

Laboration 3 – Ett API för sensordata

Uppgift

Vi började med att ni skulle använda git även under laborationerna för att öva på git efter sommaren. Från och med nu får ni välja om vill fortsätta med detta, annars - skippa git för laborationerna, men använd det när ni jobbar med IoT-caset.

Bygg ett mindre HTTP-API för sensordata som en fristående laboration. API:t ska antingen ta emot sensordata, tillgängliggöra sensordata eller göra båda delarna. Utgå gärna från lärardemot och lägg tiden på kontrakt, testning och förståelse. Erfarenheterna från laborationen används därefter i gruppens IoT-case

Kopierbara kommandon för Windows PowerShell och Linux finns i [Testkommandon för laboration 3](#)

Krav

- använd HTTP lokalt eller i ett uttryckligen tillåtet labbnät
- implementera minst en endpoint för sensordata
- använd casets befintliga fält `sensorId`, `value` och `unit`
- returnera JSON och relevanta HTTP-statuskoder
- dokumentera endpointens metod, sökväg, request och response
- testa både ett lyckat anrop och minst ett felsvar
- om ni väljer att använda Git för laborationen ska inga lösenord, tokens eller verkliga personuppgifter lagras där

Del 1 – Välj API-flöde

Välj ett av följande minsta flöden:

```
A: sensorprogram → POST /api/readings → API
B: API → GET /api/readings/latest → klient
C: sensorprogram → POST → API → GET → klient
```

Alternativ C knyter ihop båda riktningarna men är inte nödvändigt för att förstå dagens mål

Del 2 – Skriv endpoint-kontraktet först

Lägg en tabell i README eller `docs/api.md`:

Del	Exempel
Metod	<code>POST</code>
Sökväg	<code>/api/readings</code>
Content-Type	<code>application/json</code>
Request body	<code>sensorId, value, unit</code>

Del	Exempel
Lyckat svar	201 Created
Felsvar	400, 415

Använd följande datakontrakt:

```
{
  "sensorId": "temp-01",
  "value": 21.7,
  "unit": "C"
}
```

Del 3 – Implementera och starta API:t

Utgå från lärardemot eller en motsvarande C++-lösning. Gör först en hälsokontroll:

```
curl -i http://127.0.0.1:8080/health
```

Kontrollera att ni ser både statuskod och response body

Del 4 – Testa det lyckade flödet

Exempel i Linux:

```
curl -i -X POST http://127.0.0.1:8080/api/readings \
-H 'Content-Type: application/json' \
-d '{"sensorId":"temp-01","value":21.7,"unit":"C"}'
```

Exempel i PowerShell:

```
curl.exe -i -X POST http://127.0.0.1:8080/api/readings -H "Content-Type:
application/json" -d "{\"sensorId\":\"temp-01\", \"value\":21.7, \"unit\":\"C\"}"
```

Om ni implementerat läsning:

```
curl -i http://127.0.0.1:8080/api/readings/latest
```

Del 5 – Testa fel

Prova minst två av följande:

- fel sökväg
- saknad **Content-Type**
- trasig JSON
- saknat **sensorId**
- text i stället för tal i **value**
- stoppad server

Anteckna för varje test:

Test	Förväntat	Observerat	Slutsats
Saknat sensorId	400 med felmeddelande	...	...

Del 6 – För över erfarenheterna till caset

När den fristående laborationen fungerar, använd resultatet för att uppdatera gruppens case. Ni kan själva välja om ni vill använda Git under laborationen. För arbetet med caset används Git som tidigare

Uppdatera exempelvis gruppens flöde:

```
Sensor/simulator → HTTP POST → API → minne → HTTP GET → klient
```

API:t behöver inte ersätta MQTT. De kan senare vara olika delar av samma system. Lägg endast in relevanta delar i casets repository och kontrollera att inga lösenord, tokens eller verkliga personuppgifter följer med

Dagens arbetsresultat

Arbetet kan sikta mot att gruppen har:

- ett startbart API med minst en endpoint
- ett dokumenterat endpoint-kontrakt
- sparade exempel på lyckat anrop och felsvar
- en uppdaterad flödesskiss
- om ni använder Git för laborationen: en commit som beskriver API-tillägget

Fördjupning om ni blir klara

- lägg till **GET /api/readings/{sensorId}**
- lagra flera mätningar i minnet
- returnera **Location**-header efter **POST**
- skapa en OpenAPI-skiss för endpointen